

Sommario

Capitolo 12 – IPC Pipe Anonime	2
La sincronizzazione in Unix	2
Pipe Anonime	2
Pipe anonime: pipe()	2
Pipe anonime: lettura	3
Pipe anonime: scrittura	3
Pipe anonime: un singolo processo	3
Pipe anonime: utilizzo	4
Pipe half-duplex dopo una fork	4
Pipe anonima: comunicazione padre-figlio	5
Esempio 1: padre scrive – figlio legge	6
Esempio 2: leggere messaggi dal figlio	8
Comunicazione tramite pipe	8
Pipe ed exec()	10
Esempio: dup2() e sort	12
Esempio: ls sort	14
Esempio: redirectione con pipe (connect.c)	15
Implementazione delle pipeline	16
Capitolo 13 – PIPE con Nome: FIFO	16
Pipe con nome: FIFO	17
Esempio: mkfifo()	17
Accedere ad una FIFO	18
Aprire una FIFO con open()	18
Esempio	19
Esecuzione senza O_NONBLOCK	22
Esecuzione con O_NONBLOCK	22
FIFO in modalità bloccante	23
FIFO in modalità non bloccante (O_NONBLOCK)	23
Lettura e scrittura con O_NONBLOCK impostato	24
Sezione Read (lettura da FIFO vuota)	24
Sezione Write (scrittura su FIFO)	24
Esempio: comunicazione tra due processi con FIFO (produttore-consumatore)	25
Comunicazione Client-Server con FIFO	27
Gestione del Ciclo di Vita e Persistenza del Server nella Comunicazione FIFO	27

Capitolo 12 – IPC Pipe Anonime

La sincronizzazione in Unix

Nel contesto dei **sistemi operativi Unix**, la comunicazione tra processi distinti avviene attraverso meccanismi noti come **Interprocess Communication (IPC)**. Questi strumenti permettono a più processi, che possono risiedere sulla stessa macchina o su macchine differenti, di **scambiarsi dati** e **segnalare eventi**, con l'obiettivo di cooperare o sincronizzarsi.

Nel caso di processi locali, i principali mezzi di comunicazione includono **segnali**, **pipe**, **FIFO** e **socket**, mentre per la comunicazione tra macchine diverse si utilizzano prevalentemente i **socket**. La comunicazione tra processi può essere rivolta alla **cooperazione**, ovvero alla condivisione di dati per raggiungere un obiettivo comune, oppure alla **sincronizzazione**, che consente a processi indipendenti di coordinare le loro attività, evitando ad esempio accessi concorrenti a risorse condivise.

La **sincronizzazione** in Unix si articola in due aspetti fondamentali: lo **scambio di dati** e la **segnalazione di azioni**. Lo scambio di dati può avvenire tramite **pipe**, che sono operazioni di I/O su code FIFO sincronizzate dal sistema operativo; tramite **messaggi**, ovvero invio e ricezione di messaggi tipizzati; oppure attraverso la **memoria condivisa**, allocata e associata ai processi mediante chiamate di sistema. La segnalazione di eventi, invece, può avvenire tramite **segnali software** o mediante **semafori**, che costituiscono una generalizzazione dei semafori classici utilizzati per il controllo della concorrenza.

Pipe Anonime

Nel contesto dei sistemi Unix, le **pipe anonime** rappresentano un meccanismo fondamentale di **comunicazione interprocesso**. Una pipe anonima è un canale di comunicazione **unidirezionale**, mantenuto a livello **kernel**, che consente il passaggio di dati da un processo a un altro. Questa modalità è definita **half-duplex**, poiché i dati possono fluire in una sola direzione alla volta.

L'utilizzo delle pipe anonime è strettamente connesso alla **gerarchia dei processi**: solo processi che condividono un **antenato comune** possono comunicare attraverso di esse. Questo vincolo nasce dal fatto che la creazione di una pipe avviene tramite la chiamata di sistema **pipe()**, che restituisce due **descrittori di file**: uno per la lettura e uno per la scrittura. Il descrittore di lettura viene utilizzato con la funzione **read()**, mentre quello di scrittura con **write()**. I dati trasmessi vengono temporaneamente conservati in un **buffer** gestito dal kernel, la cui dimensione è limitata (indicata da **PIPE_BUF**, tipicamente 4 KB).

Quando un processo termina l'uso di una pipe, deve chiudere esplicitamente il relativo descrittore tramite la funzione **close()**. Se il **buffer è pieno**, il processo scrivente viene sospeso fino a che non vi è spazio disponibile; se invece il **buffer è vuoto**, è il processo lettore a essere sospeso in attesa di nuovi dati.

Nel quotidiano utilizzo delle shell Unix, le pipe anonime sono familiari grazie al meccanismo delle **pipeline**, che permettono di concatenare comandi, collegando l'**output standard** di un comando all'**input standard** del successivo (esempio: `cmd1 | cmd2`). La shell si occupa di creare un processo separato per ciascun comando e di stabilire il collegamento tra essi tramite pipe.

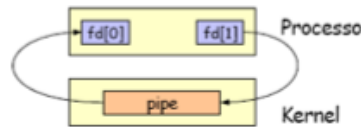
Infine, è utile distinguere le **pipe anonime**, presenti in tutte le versioni di Unix, dalle **pipe con nome (FIFO)**, introdotte in **UNIX System V**. Queste ultime, a differenza delle pipe anonime, sono identificate da un nome nel file system e possono essere utilizzate anche tra processi non imparentati.

Pipe anonime: pipe()

```
#include <unistd.h>

int pipe (int fd[2])
```

La creazione e gestione delle **pipe anonime** in Unix avviene tramite la chiamata di sistema **pipe()**, la quale genera un canale di comunicazione a livello **kernel** restituendo due **descrittori di file**: uno per la **lettura** (`fd[0]`) e uno per la **scrittura** (`fd[1]`). Se il kernel non può creare una nuova pipe, la funzione restituisce **-1**, altrimenti **0**.



Pipe anonime: lettura

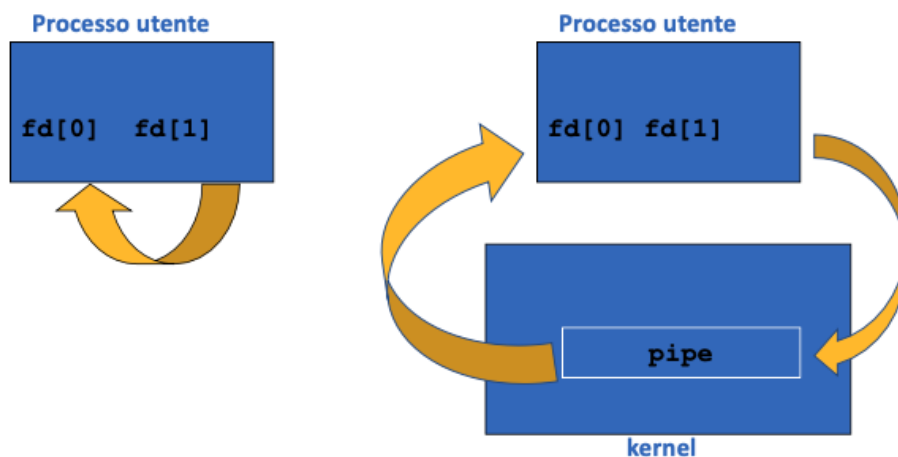
Durante la **lettura** da una pipe, la funzione **read()** si comporta in modo diverso a seconda delle condizioni: restituisce **0** se l'estremità di scrittura è stata chiusa e tutti i dati sono stati letti (indicando la **fine del flusso**); se la pipe è vuota ma la scrittura è ancora aperta, il processo **viene sospeso** in attesa di nuovi dati. Inoltre, se si tentano di leggere più byte di quelli disponibili, **read()** restituisce il **numero effettivo** di byte letti.

Pipe anonime: scrittura

Dal lato della **scrittura**, se l'estremità di lettura è chiusa, la funzione **write()** fallisce (restituisce **-1**) e viene inviato al processo scrivente un **segnale SIGPIPE**, che provoca la terminazione del processo salvo gestione diversa. Quando la quantità di dati scritta è **inferiore o uguale a PIPE_BUF**, l'operazione è **atomica**, ovvero i dati non possono essere intercalati con quelli di altri processi. Tuttavia, se i byte scritti superano **PIPE_BUF**, non è garantita l'**atomicità**, con il rischio di **interferenze tra scritture**.

È importante sottolineare che la funzione **lseek()** non può essere applicata alle pipe, in quanto non hanno una posizione di lettura/scrittura modificabile. Inoltre, l'**accesso** a una pipe anonima è limitato al **processo che la crea** e ai suoi **discendenti**, poiché il canale viene identificato tramite **descriptori di file** che non possono essere condivisi arbitrariamente tra processi non correlati.

Pipe anonime: un singolo processo



Nel caso di un **singolo processo**, una **pipe anonima** può essere utilizzata per gestire internamente il flusso di dati tra due componenti dello stesso processo, mantenendo comunque l'intermediazione del **kernel**. I due estremi della pipe, ovvero i **descriptori di file** **fd[0]** per la lettura e **fd[1]** per la scrittura, sono entrambi disponibili nello stesso **processo utente**.

In questa configurazione, il **flusso dei dati** avviene dal descrittore di scrittura verso quello di lettura, passando attraverso il **buffer del kernel**. Questo schema consente, ad esempio, di simulare una comunicazione asincrona interna, utile in situazioni di testing o nel trattamento modulare dei dati all'interno di un programma. Anche se i dati non escono mai dal processo, la loro gestione avviene comunque come se fossero trasferiti tra processi diversi, sfruttando il meccanismo delle pipe per garantire **sincronizzazione** e **bufferizzazione** automatica.

Pipe anonime: utilizzo

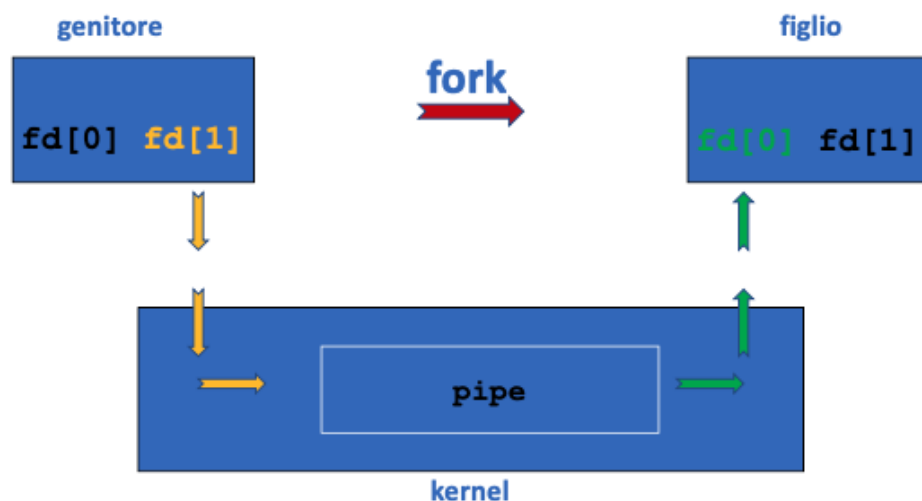
L'uso delle **pipe anonime** è strettamente legato alla comunicazione tra **processi correlati**, in particolare tra **processo padre e figlio**. Sebbene sia tecnicamente possibile utilizzare una pipe in un **processo singolo**, ciò ha scarsa utilità pratica. Infatti, il contesto tipico prevede che il processo che crea la pipe invochi **fork()**, dando origine a un secondo processo che eredita i **descrittori di file** creati dalla chiamata a **pipe()**.

La **sequenza classica** di utilizzo è la seguente: prima si invoca **pipe()** per creare la pipe, poi **fork()** per generare il processo figlio. Dopo la fork, ogni processo **chiude** l'estremità della pipe che non utilizza (il lettore chiude la scrittura e viceversa), e la comunicazione può avvenire attraverso le chiamate **write()** e **read()**. Una volta conclusa la comunicazione, ogni processo **chiude** il proprio descrittore con **close()**.

Nel caso in cui si voglia ottenere una **comunicazione bidirezionale**, non basta una singola pipe poiché è **unidirezionale**: è necessario creare **due pipe**, una per ciascun verso del flusso di dati.

Inoltre, la **chiamata fstat()** consente di determinare il tipo di file associato a un descrittore di file di una pipe, che viene riconosciuto come **FIFO**, e ciò può essere verificato con la macro **S_ISFIFO**. Questo approccio rafforza l'importanza del legame tra **pipe anonime** e il **modello gerarchico dei processi**, in cui il passaggio dei descrittori avviene al momento della fork.

Pipe half-duplex dopo una fork



L'immagine illustra il funzionamento di una **pipe half-duplex** dopo una **fork()**, che rappresenta il tipico caso d'uso delle **pipe anonime** per la comunicazione tra processi.

In questo scenario, il **processo genitore** crea inizialmente la pipe, ottenendo i due **descrittori di file** **fd[0]** (per la lettura) e **fd[1]** (per la scrittura). Al momento della **fork**, il processo **figlio eredita entrambi i descrittori**, rendendo possibile la comunicazione.

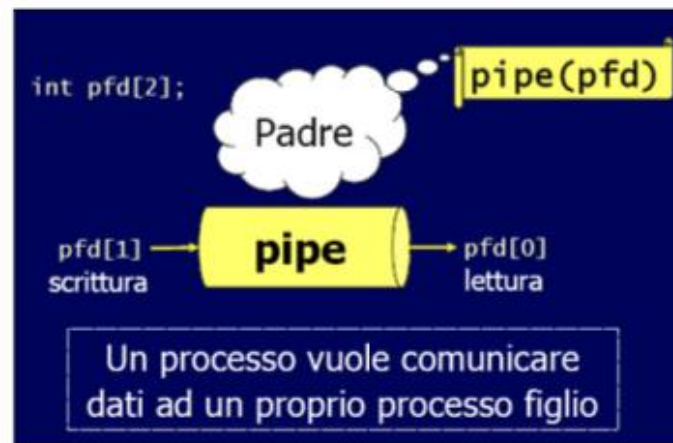
Per realizzare un corretto flusso **unidirezionale di dati** (half-duplex), è necessario che ogni processo **chiuda** l'estremità della pipe che non utilizza: ad esempio, il genitore può scrivere tramite **fd[1]** e chiudere **fd[0]**, mentre il figlio legge da **fd[0]** e chiude **fd[1]**.

Viceversa, se si desidera inviare dati **dal figlio al genitore**, sarà il genitore a **chiudere fd[1]** e il figlio a **chiudere fd[0]**, in modo da utilizzare correttamente i canali opposti della pipe.

Il **kernel** funge da intermediario, trasferendo i dati scritti in **fd[1]** nel **buffer della pipe**, da cui verranno successivamente letti tramite **fd[0]**.

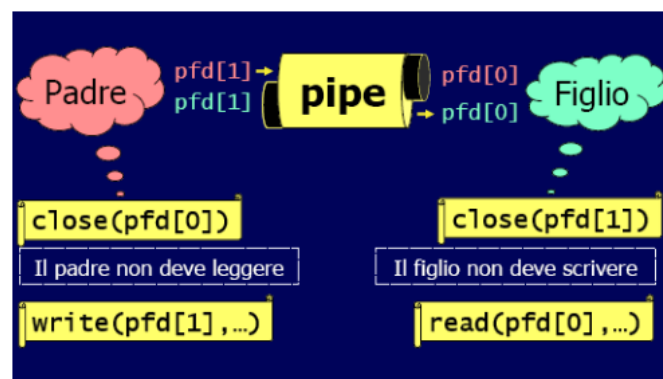
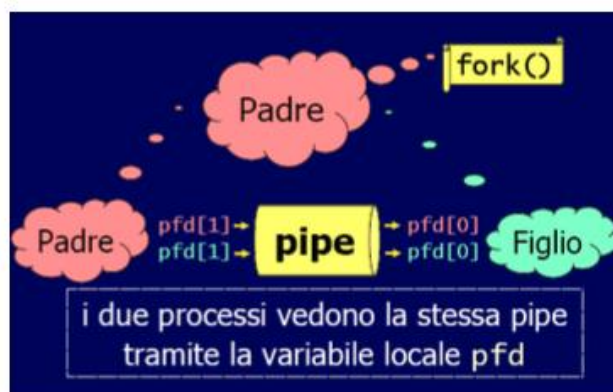
Questo schema permette un **canale di comunicazione affidabile e sincronizzato** dal processo genitore al figlio (o viceversa), ed è alla base di molte implementazioni di meccanismi IPC nei sistemi Unix.

Pipe anonima: comunicazione padre-figlio

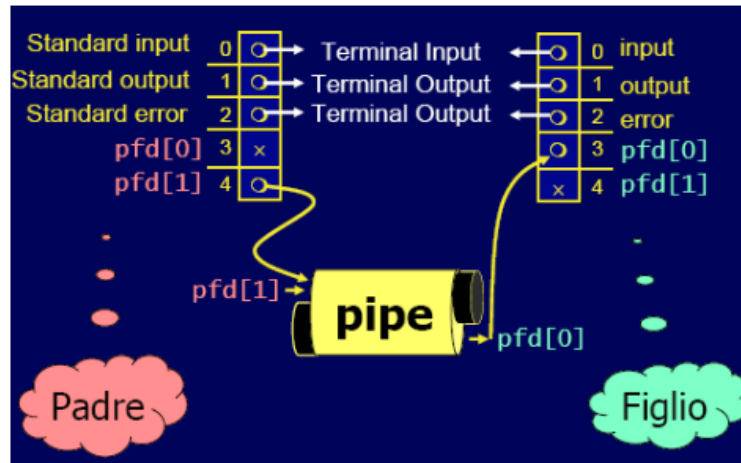


Le immagini illustrano chiaramente il processo di **comunicazione tramite pipe anonime** tra un **processo padre** e un **processo figlio**, fornendo una visione pratica di come i **descriptori di file** e la **tabella dei descriptori** vengano gestiti dal sistema.

Il processo inizia con la chiamata a **pipe(pfd)**, dove viene creata una **pipe anonima** i cui estremi sono `pfd[0]` (per la lettura) e `pfd[1]` (per la scrittura). Subito dopo, il processo padre invoca **fork()**, generando un **processo figlio** che eredita una **copia esatta della tabella dei descriptori** del padre, inclusi quelli della pipe. A questo punto, **entrambi i processi condividono la stessa pipe**, identificata dalla **variabile locale pfd**.



Per realizzare una **comunicazione direzionale**, ad esempio **dal padre al figlio**, è necessario che il padre **chiuda** `pfd[0]` (non deve leggere) e il figlio **chiuda** `pfd[1]` (non deve scrivere). La trasmissione dei dati avviene quindi attraverso `write(pfd[1], ...)` da parte del padre e `read(pfd[0], ...)` da parte del figlio.



Infine, la rappresentazione della **tabella dei descrittori** mostra come i descrittori standard (input, output, errore) e quelli relativi alla pipe vengano gestiti in modo indipendente, ciascuno con un proprio **indice numerico** (ad esempio 3 e 4 per pfd[0] e pfd[1]). Questa gestione consente al sistema operativo di distinguere tra i vari flussi di I/O e instradare correttamente i dati.

Questo modello è alla base di moltissimi meccanismi di **comunicazione interprocesso (IPC)** in ambiente Unix, specialmente in contesti in cui si utilizzano **pipeline di comandi**, come avviene comunemente nelle shell.

Esempio 1: padre scrive – figlio legge

```
#include <string.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#define SIZE 1024
int main(int argc, char*argv[]) {
    int pfd[2];
    int nread;
    int pid;
    char buf[SIZE];
    if (pipe(pfd) == -1) {
        perror("pipe() fallita");
        exit(-1);
    }
    if ((pid=fork()) < 0 ) {
        perror("fork() fallita");
        exit(-2);
    }
    ...
}
```

```

...
if(pid==0) { /* figlio */
    close(pfd[1]);
    while ( (nread=read(pfd[0], buf, SIZE)) != 0)
        printf("il figlio legge: %s\n", buf);
    close(pfd[0]);
} else { /* padre */
    close(pfd[0]);
    strcpy(buf, "Sono tuo padre!");
    write(pfd[1], buf, strlen(buf)+1);
    close(pfd[1]);
}
exit(0);
}

```

L'esempio di codice mostrato rappresenta in modo pratico il classico scenario di comunicazione tramite **pipe anonima** tra un **processo padre** e un **processo figlio**, dove il padre **scrive** e il figlio **legge**.

Il programma inizia includendo le librerie necessarie e definendo una costante `SIZE` per il buffer. Viene poi dichiarato un array `pfd[2]` per contenere i due **descrittori di file** della pipe. Dopo la creazione della pipe con `pipe(pfd)`, viene invocata la **fork()**, che genera un processo figlio. Entrambi i processi ereditano i descrittori `pfd[0]` e `pfd[1]`.

Nel **processo figlio**, viene chiusa l'estremità di scrittura `pfd[1]`, in quanto non necessaria. Il figlio legge dalla pipe usando un ciclo con `read(pfd[0], ...)` fino a che non riceve **EOF** (cioè `read()` restituisce 0), e stampa il messaggio ricevuto. Al termine, chiude anche `pfd[0]`.

Nel **processo padre**, invece, viene chiusa l'estremità di lettura `pfd[0]`. Il padre copia nel buffer la stringa "Sono tuo padre!", quindi scrive nella pipe usando `write(pfd[1], ...)`. Si assicura di includere anche il carattere di fine stringa (`\0`) scrivendo `strlen(buf)+1`.

`strlen(buf) + 1` garantisce che venga trasmessa una **stringa ben formata**, completa di terminatore '`\0`', essenziale per l'uso corretto in C.

Infine, chiude `pfd[1]` per segnalare al figlio che la scrittura è conclusa.

Questa implementazione mostra in modo chiaro la corretta gestione dei descrittori e la **sincronizzazione implicita** che le pipe garantiscono, grazie alla **bufferizzazione** e alla **chiusura controllata** delle estremità. È un esempio didattico efficace per comprendere il funzionamento della **IPC unidirezionale** tra processi correlati in Unix.

Esempio 2: leggere messaggi dal figlio

```
/* talk.c */
#include <stdio.h>
#include <string.h>
#define READ 0
#define WRITE 1
char *frase = "Messaggio...";
int main() {
    int fd[2], bytesRead;
    char message[100];

    pipe(fd);
    if(fork() == 0) {
        close(fd[READ]);
        write(fd[WRITE], frase, strlen(frase)+1);
        close(fd[WRITE]);
    }

    else {
        close(fd[WRITE]);
        bytesRead=read(fd[READ], message,
            100);
        printf("Letti %d byte: %s\n",
            bytesRead, message);
        close(fd[READ]);
    }
}

$ a.out
Letti 12 byte: Messaggio...
$
```

Questo secondo esempio mostra una comunicazione **unidirezionale tramite pipe anonima**, in cui è il **figlio** a inviare un messaggio al **padre**. Il programma definisce due costanti, **READ** e **WRITE**, per rappresentare i due estremi della pipe, corrispondenti rispettivamente a `fd[0]` e `fd[1]`.

Dopo la creazione della **pipe** con la chiamata `pipe(fd)`, viene eseguita una **fork()**. Il processo figlio chiude l'estremità destinata alla **lettura** (`fd[READ]`) e utilizza `write()` per inviare il contenuto della stringa "Messaggio..." attraverso la pipe. È importante notare che viene scritto anche il **carattere di terminazione** (`'\0'`), grazie all'uso di `strlen(frase) + 1`, in modo che il padre possa interpretare correttamente il messaggio come una **stringa in C**. Dopo l'invio del messaggio, il figlio chiude l'estremità di **scrittura**.

Il processo padre, nel frattempo, chiude la sua estremità di **scrittura** (`fd[WRITE]`) e utilizza `read()` per ricevere i dati dalla pipe. Il messaggio ricevuto viene memorizzato nel buffer `message`, e successivamente stampato a video insieme al numero di **byte letti**. Anche il padre, al termine della lettura, chiude la sua estremità di **lettura**.

Il corretto funzionamento si riflette nell'output finale, in cui il padre conferma di aver ricevuto **12 byte**, seguiti dalla stampa del **messaggio completo**. Questo esempio evidenzia chiaramente l'uso delle **pipe anonime** per la **comunicazione tra processi imparentati**, e dimostra come la gestione accurata dei **descrittori di file** e dei **terminatori di stringa** sia cruciale per garantire uno scambio di dati affidabile e ben formato.

Comunicazione tramite pipe

Quando un processo **scrittore** invia attraverso una **pipe** più messaggi di **lunghezza variabile**, è fondamentale stabilire un **protocollo di comunicazione** che consenta al processo **lettore** di riconoscere correttamente dove termina ciascun messaggio. In assenza di questo meccanismo, il lettore non può sapere quanti byte leggere per ottenere un messaggio completo, rischiando così di leggere messaggi parziali o dati concatenati in modo errato.

Una strategia consiste nell'inviare, prima del messaggio vero e proprio, la sua **lunghezza** codificata in un numero di byte **fisso e noto**, che il lettore usa per sapere quanti byte successivi leggere. Un'altra possibilità è quella di terminare ciascun messaggio con un **carattere speciale**, come il **carattere nullo** `'\0'` o un **a capo** (`newline`), che funge da delimitatore naturale.

In termini più generali, il **protocollo** adottato deve definire chiaramente la **sequenza e il formato dei messaggi** che le due parti si scambieranno, in modo che la comunicazione rimanga **sincrona, coerente e interpretabile** da entrambi i lati. Questo tipo di accordo implicito o esplicito è essenziale per il corretto funzionamento delle comunicazioni basate su **pipe anonime**.

Protocol.c (Semplice Protocollo)

```
/* Semplice protocollo di comunicazione tramite pipe anonima. Dovendo inviare messaggi di lunghezza
variabile ogni messaggio è preceduto da un intero che fornisce la sua lunghezza in caratteri */

#define READ 0 /* Estremità in lettura della pipe */
#define WRITE 1 /* Estremità in scrittura della pipe */

char *msg[3] = { "Primo", "Secondo", "Terzo" };

int main (void) {
    int fd[2], i, length, bytesRead;
    char buffer [100]; /* Buffer del messaggio */
    pipe (fd); /* Crea una pipe anonima */
    if (fork () == 0) { /* Figlio, scrittore */
        close(fd[READ]); /* Chiude l'estremità inutilizzata */
        for (i = 0; i < 3; i++) {
            length=strlen (msg[i])+1 ; /* include \0 */
            write (fd[WRITE], &length, sizeof(int));
            write (fd[WRITE], msg[i], length); }
        close (fd[WRITE]); /* Chiude l'estremità usata */
    }

    else { /* Genitore, lettore */
        close (fd[WRITE]); /* Chiude l'estremità non usata */
        while (read(fd[READ], &length, sizeof(int))) {
            bytesRead = read (fd[READ], buffer, length);
            if (bytesRead != length) {
                printf("Errore!\n");
                exit(1);
            }
            printf("Padre:Letti %d byte:%s\n",bytesRead,buffer);
        }
        close (fd[READ]); /* Chiude l'estremità usata */
    }
    exit(0);
}
```

Questo esempio mostra un **semplice protocollo di comunicazione** tramite **pipe anonima** per l'invio di **messaggi di lunghezza variabile**. L'idea centrale è quella di far precedere ogni messaggio da un **intero che rappresenta la sua lunghezza**, in modo che il processo **lettore** sappia esattamente quanti byte leggere.

Nel **processo figlio**, che funge da **scrittore**, viene chiusa l'estremità di lettura della pipe. Per ciascuno dei tre messaggi contenuti nell'array `msg`, viene calcolata la **lunghezza**, includendo anche il **carattere di terminazione '\0'**, così da inviare una **stringa ben formata**. Ogni messaggio viene inviato in due fasi distinte: prima viene scritto l'intero `length` usando `write()` con `sizeof(int)`, e subito dopo il contenuto effettivo del messaggio. Al termine, viene chiusa anche l'estremità di scrittura.

Il **processo padre**, che funge da **lettore**, chiude l'estremità di scrittura della pipe. Entra poi in un ciclo in cui legge inizialmente l'intero che indica la **lunghezza del messaggio**, e subito dopo legge esattamente `length` byte dalla pipe. Se la quantità di byte letti non corrisponde alla lunghezza prevista, viene stampato un **errore** e il programma termina. In caso contrario, il messaggio viene visualizzato. Anche il lettore, infine, chiude la propria estremità della pipe.

Questo esempio evidenzia chiaramente come un **protocollo esplicito**, basato sull'invio anticipato della **lunghezza del messaggio**, consenta di gestire con precisione flussi di dati **variabili**, evitando problemi di delimitazione e lettura parziale. È un approccio semplice, ma robusto, per la **comunicazione sincrona** tra processi in ambiente Unix.

Pipe ed exec()

Quando si utilizza una **pipe** per collegare due processi, è fondamentale che **entrambi i processi conoscano i descrittori di file** associati alla pipe stessa. Finora, questo è stato possibile grazie al fatto che, in seguito a una **fork()**, il **processo figlio eredita la tabella dei descrittori** del padre, permettendo una comunicazione diretta tramite le **variabili locali** contenenti `fd[0]` e `fd[1]`.

Tuttavia, nel momento in cui uno dei due processi esegue una **exec()**, l'intero **spazio di memoria del processo** viene sostituito, comprese le variabili locali. Questo comporta la perdita dei riferimenti ai descrittori, anche se la **tabella dei descrittori di file** del processo rimane **inalterata**. In altre parole, i descrittori esistono ancora, ma non sono più accessibili se non vengono gestiti correttamente prima dell'`exec()`.

Un esempio pratico è la realizzazione di una **pipeline** tra due comandi, come `ls | wc -w`. Per realizzarla manualmente, un processo (il padre) crea una pipe, poi esegue **fork()**, e infine entrambi i processi eseguono **exec()** per trasformarsi rispettivamente in `ls` e `wc -w`. La sfida è che, una volta eseguito `exec()`, non si ha più accesso alle variabili locali che contenevano i descrittori della pipe, pur essendo quei descrittori ancora validi e aperti.

Questa situazione evidenzia la necessità di **gestire correttamente i descrittori** prima dell'esecuzione di un nuovo programma con `exec()`, ad esempio usando **dup2()** per collegare i descrittori della pipe agli **standard input/output** del processo, così da assicurarsi che la comunicazione via pipe sia ancora possibile anche dopo il cambio di immagine di processo.

Esempio

```
#include <stdio.h>
#define SIZE 1024
int main(int argc, char** argv) {
    int pfd[2], pid;
    if (pipe(pfd) == -1) {
        perror("pipe() failed");
        exit(-1);
    }
    if ((pid=fork()) < 0) {
        perror("fork() failed");
        exit(-2);
    }
    ...

    ...
    if(pid==0) { /* figlio */
        close(pfd[1]);
        dup2(pfd[0], 0);
        close(pfd[0]);
        execlp("wc", "wc", "-w", NULL);
        perror("wc fallita");
        exit(-3);
    } else { /* padre */
        close(pfd[0]);
        dup2(pfd[1], 1);
        close(pfd[1]);
        execlp("ls", "ls", NULL);
        perror("ls fallita");
        exit(-4);
    }
    exit(0);
}
```

Questo **Esempio** mostra come implementare una **pipeline tra due comandi** (`ls | wc -w`) utilizzando **pipe anonime**, **fork()** e **exec()**, replicando il comportamento tipico della shell. Si tratta di un caso di **comunicazione interprocesso** dove il padre e il figlio eseguono comandi diversi ma comunicano tramite una pipe.

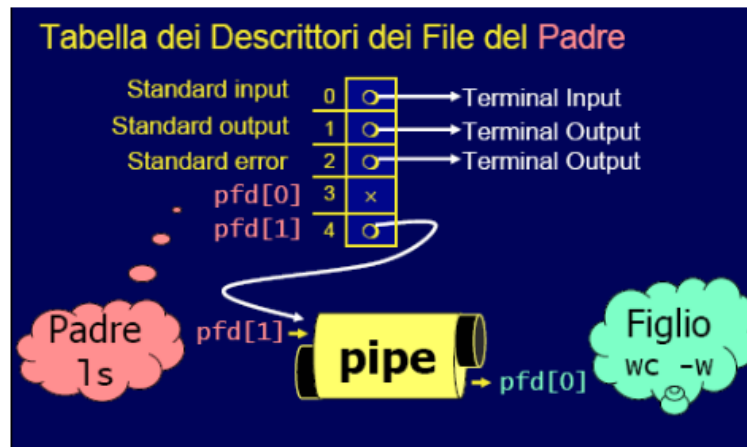
Il programma crea una pipe (`pfd[0]` per la **lettura**, `pfd[1]` per la **scrittura**) e invoca **fork()**. Entrambi i processi ereditano la pipe, ma si preparano a usarla in direzioni opposte.

Nel **processo figlio**, che dovrà eseguire il comando `wc -w`, viene chiusa l'estremità di scrittura `pfd[1]`, poiché il figlio deve solo **leggere**. Con **dup2(pfd[0], 0)**, il descrittore `pfd[0]` viene duplicato sull'**input standard** (file descriptor 0). In questo modo, il comando eseguito tramite **execlp()** leggerà dalla pipe come se stesse leggendo dall'input standard. Subito dopo, `pfd[0]` viene chiuso e il processo viene sostituito da `wc -w`.

Nel **processo padre**, destinato a eseguire `ls`, viene chiusa l'estremità di lettura `pfd[0]`, e `pfd[1]` viene duplicato su **stdout** con **dup2(pfd[1], 1)**. In questo modo, tutto ciò che `ls` scriverà sul suo output standard sarà in realtà inviato nella pipe. Anche qui, dopo la duplicazione, il descrittore originale viene chiuso, e il processo esegue `ls` tramite **execlp()**.

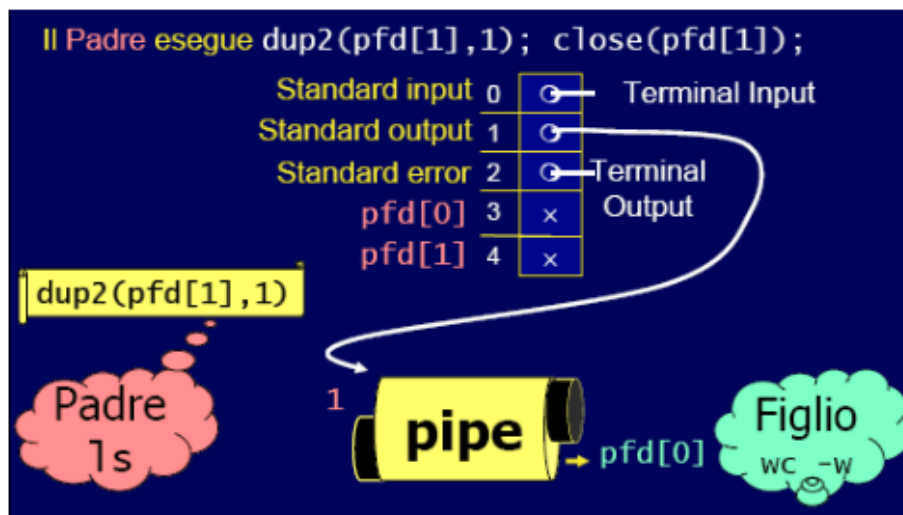
Questa tecnica è fondamentale perché, dopo `exec()`, tutte le **variabili locali** (incluso l'array `pfid`) vengono perse, ma i **descrittori duplicati** restano validi e attivi nella **tabella dei descrittori**. Grazie a questa preparazione, i due processi riescono a **comunicare tramite la pipe anche dopo aver eseguito programmi completamente diversi**. Si tratta quindi di un esempio concreto e molto potente di **composizione di comandi tramite IPC** nei sistemi Unix.

dup2 e pipelining

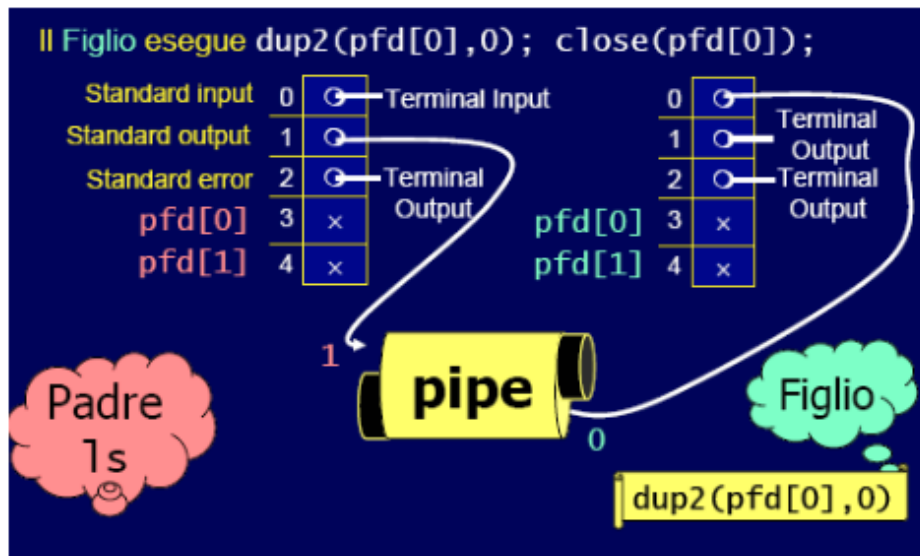


Questa sequenza di immagini illustra in modo chiaro il ruolo della funzione `dup2()` nel processo di **creazione di pipeline tra comandi** in Unix, utilizzando **pipe anonime**, `fork()`, e `exec()`.

Dopo la creazione della **pipe** con `pipe(pfd)`, il processo padre esegue `fork()` per generare il processo figlio. Entrambi i processi ereditano i **descrittori di file** `pfd[0]` (lettura) e `pfd[1]` (scrittura).

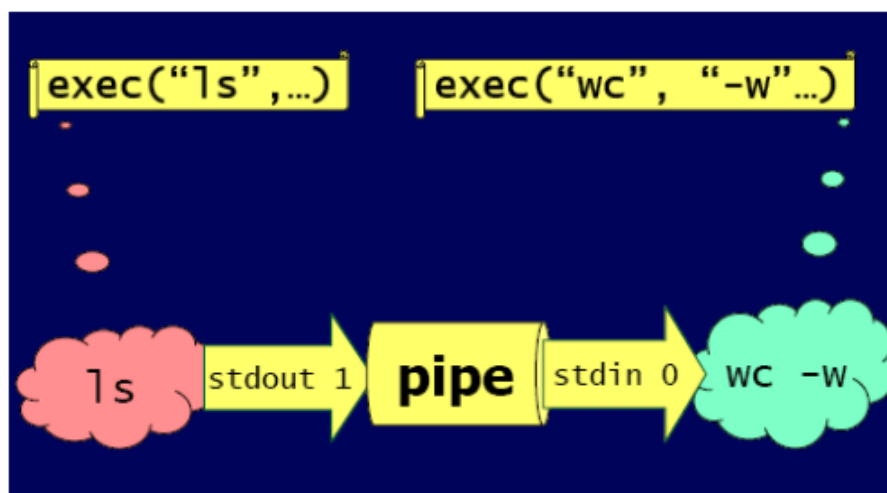


Nel **padre**, destinato a eseguire il comando `ls`, si vuole che l'**output standard** (file descriptor 1) sia collegato alla **scrittura della pipe**. Per farlo, si utilizza `dup2(pfd[1], 1)`, che fa sì che tutto ciò che `ls` stampa venga scritto nella pipe.



Dopo la duplicazione, `pfd[1]` viene chiuso perché non più necessario.

Nel **figlio**, che eseguirà il comando `wc -w`, si desidera che il suo **input standard** (file descriptor 0) legga dalla pipe. Questo avviene tramite **`dup2(pfd[0], 0)`**, che collega l'estremità di lettura della pipe allo standard input. Anche qui, il descrittore `pfd[0]` viene poi chiuso.



Infine, entrambi i processi eseguono un **`exec()`**, che sostituisce l'immagine di processo corrente con quella del comando desiderato. I **descrittori duplicati** sopravvivono alla `exec()` e permettono il **flusso continuo dei dati**: l'output di `ls` entra nella pipe e diventa l'input di `wc -w`.

Esempio: `dup2()` e `sort`

Questo **Esempio 4** mostra come utilizzare le **pipe anonime** in combinazione con **`dup2()`** per inviare dati da un processo padre a un comando esterno, in questo caso **`sort`**, eseguito dal figlio. Si tratta di un classico caso di **redirect dello standard input** tramite pipe.

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
int main ()
{
    int fds[2];
    pid_t pid;
    /* Crea una pipe. I descrittori per le estremità sono in fds.*/
    pipe (fds);
    /* Crea un processo figlio. */
    pid = fork ();
    if (pid == (pid_t) 0) {
        /* Processo figlio. Chiude descrittore lato scrittura */
        close (fds[1]);
        /* Connette il lato lettura della pipe allo standard input. */
        dup2 (fds[0], STDIN_FILENO);
        /* Sostituisce il processo figlio con il programma "sort". */
        execlp ("sort", "sort", 0);
    }
}

```

Dopo aver creato la **pipe** (`fds[0]` per la lettura, `fds[1]` per la scrittura), il programma invoca `fork()`, generando un **processo figlio**. Il **figlio** chiude l'estremità di scrittura della pipe (`fds[1]`) e con `dup2(fds[0], STDIN_FILENO)` reindirizza il proprio **standard input** verso l'estremità di lettura della pipe. A questo punto, il figlio esegue il comando `sort` tramite `execlp()`. Così facendo, il comando `sort` leggerà i dati direttamente dalla pipe, come se arrivassero dalla tastiera.

```

else { /* Processo genitore */
    /* Chiude il descrittore lato lettura */
    close (fds[0]);
    write(fds[1], "This is a test.\n", 16);
    write(fds[1], "Hello, world.\n", 14);
    write(fds[1], "My dog has fleas.\n", 18);
    write(fds[1], "This program is great.\n", 23);
    write(fds[1], "One fish, two fish.\n", 20);

    close (fds[1]);
    /* Aspetta che il processo figlio finisca */
    waitpid (pid, NULL, 0);
}
return 0;
}

```

Il **padre**, invece, chiude l'estremità di lettura (`fds[0]`) e utilizza `write()` per inviare una serie di **stringhe di testo** nella pipe, che rappresentano righe da ordinare. Dopo aver scritto tutte le righe, chiude anche la scrittura con `close(fds[1])`, segnalando al lettore (cioè `sort`) che non ci sono altri dati. Infine, attende la terminazione del figlio con `waitpid()`.

Questo esempio dimostra l'uso pratico di `dup2()` per implementare un **meccanismo di input controllato** per un comando eseguito tramite `exec()`. In questo modo, il **processo figlio** esegue un comando esterno leggendo i dati preparati dal padre, sfruttando la **flessibilità della tabella dei descrittori** per simulare una pipeline.

Esempio: ls | sort

L'**Esempio 5** rappresenta un caso classico di utilizzo delle **pipe anonime** per implementare una pipeline tra due comandi Unix: `ls | sort`, ovvero il risultato del comando `ls` viene inviato direttamente come input al comando `sort`.

```
int main()
{
    int pid;
    int fd[2];
    pipe(fd);
    if ((pid = fork()) == 0)
    { /* figlio */
        /* chiusura lettura da pipe */
        close(fd[0]);
        /* redirezione stdout a pipe */
        dup2(fd[1], 1);
        execlp("ls", "ls", NULL);
    } else if (pid > 0)
    { /* padre */
        /* chiusura scrittura su pipe */
        close(fd[1]); /* redirezione stdin a pipe */
        dup2(fd[0], 0);
        execlp("sort", "sort", NULL);
    }
}
```

Dopo aver creato la **pipe** con `pipe (fd)`, il programma esegue una **fork()** per creare un **processo figlio**.

Nel **figlio**, che eseguirà `ls`, viene chiuso l'estremo di **lettura** della pipe (`fd[0]`), perché non deve leggere. Successivamente, con `dup2 (fd[1] , 1)`, si duplica l'estremità di scrittura della pipe sul **file descriptor 1**, cioè lo **standard output**. In questo modo, tutto ciò che `ls` stampa andrà nella pipe. Infine, il figlio viene trasformato nel comando `ls` con `execlp ()`.

Nel **padre**, che eseguirà `sort`, viene chiuso l'estremo di **scrittura** della pipe (`fd[1]`), perché il padre deve solo leggere. Con `dup2 (fd[0] , 0)`, si duplica l'estremità di lettura sul **file descriptor 0**, cioè lo **standard input**. Questo fa sì che `sort` legga direttamente dalla pipe, ovvero dai dati emessi da `ls`. Il processo viene quindi trasformato nel comando `sort` tramite `execlp()`.

Questo esempio è estremamente efficace per mostrare come, attraverso **dup2** e **pipe**, sia possibile costruire **pipeline personalizzate** in C replicando il comportamento della shell Unix, mantenendo la separazione tra i processi ma permettendo loro di **comunicare tramite stream standard**.

Esempio: redirectione con pipe (connect.c)

Questo **esempio (connect.c)** mostra un'applicazione generica della comunicazione tra processi tramite **pipe anonime**, in cui viene realizzato un collegamento tra due **programmi esterni**, passati come **argomenti della riga di comando**. Il comportamento ottenuto è equivalente a quello di una **pipeline shell**, del tipo:

```
connect cmd1 cmd2
```

dove l'**output di cmd1** viene **reindirizzato come input di cmd2**.

```
#include <stdio.h>
#define READ 0
#define WRITE 1
int main (int argc, char *argv []) {
    int fd [2];
    pipe (fd); /* Crea una pipe senza nome */
    if (fork () != 0) { /* Padre, scrittore */
        close (fd[READ]); /* Chiude l'estremità non usata */
        dup2 (fd[WRITE], 1); /* Duplica l'estremità usata allo stdout */
        close (fd[WRITE]); /* Chiude l'estremità originale usata */
        execlp (argv[1], argv[1], NULL); /* Esegue il programma scrittore */
        perror ("connect"); /* Non dovrebbe essere eseguita */
    }
    else { /* Figlio, lettore */
        close (fd[WRITE]); /* Chiude l'estremità non usata */
        dup2 (fd[READ], 0); /* Duplica l'estremità usata allo stdin */
        close (fd[READ]); /* Chiude l'estremità originale usata */
        execlp (argv[2], argv[2], NULL); /* Esegue il programma lettore */
        perror ("connect"); /* Non dovrebbe essere eseguita */
    }
}
```

Dopo la creazione della pipe (`pipe (fd)`), viene eseguita una **fork()**. Il **padre**, che si occupa del comando **scrittore** (`cmd1`), chiude l'estremità **di lettura** della pipe (`fd[READ]`) e tramite **dup2 (fd[WRITE], 1)** collega l'estremità di **scrittura** della pipe allo **standard output**. Questo fa sì che tutto ciò che `cmd1` scrive venga inviato nella pipe. Dopo la duplicazione, chiude `fd[WRITE]` e invoca **execlp ()** per eseguire il primo comando (`argv[1]`).

Il **figlio**, che deve eseguire il comando **lettore** (`cmd2`), chiude l'estremità **di scrittura** della pipe (`fd[WRITE]`) e con **dup2 (fd[READ], 0)** collega l'estremità **di lettura** della pipe allo **standard input**. In questo modo, `cmd2` riceverà i dati come se provenissero dalla tastiera. Dopo la duplicazione, chiude `fd[READ]` e invoca **execlp ()** con il secondo comando (`argv[2]`).

Entrambi i processi terminano con `execlp ()` e quindi non dovrebbero mai eseguire il codice che segue (da qui i messaggi "non dovrebbe essere eseguita").

Questo esempio è estremamente utile per comprendere come costruire un **collegamento flessibile tra due programmi arbitrari** sfruttando la **ereditarietà dei descrittori di file** tra padre e figlio, e l'uso mirato di **dup2()** per costruire pipeline personalizzate a livello di sistema.

Implementazione delle pipeline

La logica generale dell'implementazione delle pipeline nei sistemi Unix prevede un meccanismo che consente l'esecuzione concatenata di più comandi dove **l'output di uno diventa l'input del successivo**.

Il **processo padre** svolge il ruolo di coordinatore. In primo luogo, crea un numero di **pipe** pari al numero di **connessioni necessarie**, ovvero uno in meno rispetto al numero totale di comandi da eseguire. Successivamente, genera un numero di **processi figli** pari al numero dei comandi, assegnando a ciascuno l'esecuzione di uno dei comandi previsti. Dopo aver creato le pipe e i figli, il padre **chiude tutte le estremità** delle pipe, poiché non deve partecipare alla comunicazione diretta. Infine, **attende la terminazione** di tutti i figli per garantire il completamento della pipeline.

Ogni **processo figlio**, da parte sua, chiude tutte le estremità delle pipe **che non utilizza** per evitare interferenze o letture/scritture accidentali. Successivamente, tramite **dup2()**, collega le estremità della pipe che usa agli **stream standard** appropriati (`stdin` o `stdout`). Infine, invoca una funzione della famiglia **exec()** per **sostituire se stesso con il comando da eseguire**, mantenendo attivi solo i descrittori corretti, così da consentire un flusso corretto dei dati lungo la pipeline.

Questa strategia rappresenta il **modello classico di esecuzione di pipeline**, come quelle implementate dalla shell (`ls | grep txt | sort`), ma può essere estesa a qualsiasi programma che necessiti di concatenare l'output e l'input di più processi in modo controllato ed efficiente.

Capitolo 13 – PIPE con Nome: FIFO

Pipe con nome: FIFO

Con le pipe anonime è stato possibile scambiare dati solo tra **processi imparentati**, ovvero avviati da un comune processo padre. Questa caratteristica rappresenta una **limitazione**, poiché può essere necessario permettere la comunicazione anche tra **processi indipendenti**. A tal fine, vengono introdotte le **pipe con nome**, o **FIFO**, che permettono il collegamento tra due o più processi qualsiasi. Una FIFO è un tipo speciale di **file** che si comporta come una pipe tradizionale, ma con la particolarità di avere un **nome nel file system**, permettendone così l'accesso anche a processi non correlati.

```
#include <sys/stat.h>
int mkfifo(const char *filename, mode_t mode);

/* restituisce 0 se OK, -1 in caso di errore */
```

La creazione di una FIFO può avvenire sia dalla **linea di comando** utilizzando il comando `mkfifo filename`, sia da **programma**, tramite la funzione `mkfifo()` inclusa nella libreria `<sys/stat.h>`. Questa funzione restituisce **0 in caso di successo** e **-1 in caso di errore**.

È importante notare che l'uso di `mkfifo()` implica la presenza del flag **O_CREAT|O_EXCL**, il che significa che tenterà di creare una nuova FIFO e genererà un **errore EEXIST** se il file esiste già. Se non si vuole creare una nuova FIFO, ma solo accedere a una esistente, è preferibile usare **open()** invece di `mkfifo()`. Un approccio corretto per gestire entrambi i casi consiste nell'invocare `mkfifo()`, controllare se si verifica un errore **EEXIST** e, in tal caso, procedere con **open()**.

Esempio: mkfifo()

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
int main(){
    int res = mkfifo("/tmp/my_fifo", 0777);
    if (res == 0)
        printf("FIFO creata\n");
    exit(0);
}
```

L'esempio proposto illustra come utilizzare la funzione **mkfifo()** per creare un **file speciale di tipo FIFO**. Il codice C fornito importa le librerie necessarie e crea una FIFO chiamata `my_fifo` nella directory `/tmp`, con **permessi 0777**. Se la creazione ha successo, viene stampato un messaggio a conferma.

Anche se nel codice si specificano i permessi **0777**, questi vengono **modificati dalla maschera dell'utente (umask)**, come accade nella normale creazione di file. Per esempio, con una `umask` pari a **022**, i permessi effettivi della FIFO saranno **755**. Questo comportamento riflette la gestione standard dei permessi nel file system.

La FIFO creata può essere eliminata come un qualsiasi file, sia tramite il comando **rm**, sia programmaticamente con la system call **unlink()**. Infine, dopo l'esecuzione del programma, è possibile verificare l'esistenza e i permessi della FIFO con un semplice comando **ls -lF**, che mostrerà un file di tipo pipe (`p`) con i permessi e l'utente associato.

Accedere ad una FIFO

Le pipe con nome, o **FIFO**, offrono un'importante caratteristica: risiedono nel **file system**, e quindi possono essere utilizzate nei comandi come se fossero normali **file**. Questo permette, ad esempio, di leggere e scrivere su una FIFO utilizzando comandi standard come `cat` o `echo`.

- Proviamo a leggere dalla FIFO (vuota)

```
$ cat < /tmp/my_fifo
```

Nel caso in cui si tenti di leggere da una FIFO con il comando `cat` e la FIFO sia **vuota**, il comando si **blocca** in attesa che vengano scritti dei dati. Questo comportamento dimostra come la FIFO sia effettivamente un canale di comunicazione **sincrono**, che richiede la **presenza simultanea** di lettore e scrittore per completare la comunicazione.

- Ora scriviamo nella FIFO (usando un altro terminale, infatti il primo comando si sospende in attesa di dati scritti nella FIFO)

```
$ echo "sdsdfasd" > /tmp/my_fifo
```

Scrivendo dati nella FIFO tramite il comando `echo`, il processo di lettura (`cat`) riceve immediatamente i dati e li stampa su **stdout**. Se non si invia alcun dato alla FIFO, il comando `cat` sospenderà fino a che non lo si interrompe (Ctrl + C).

```
$ cat < /tmp/my_fifo &  
[1] 1316  
$ echo "sdsdfasd" > /tmp/my_fifo  
sdsdfasd
```

È anche possibile gestire tutto da un solo terminale eseguendo `cat` in **background**, e successivamente scrivendo con `echo`. In questo scenario, `cat` viene inizialmente sospeso, ma riprende l'esecuzione non appena i dati diventano disponibili.

Una volta che il comando `echo` ha completato l'invio dei dati e la FIFO è **chiusa**, `cat` termina senza attendere ulteriori dati. Questo avviene perché la chiusura della FIFO da parte del processo scrittore segnala la fine del flusso, permettendo al processo lettore di **concludersi correttamente**.

Aprire una FIFO con `open()`

Quando si accede a una **FIFO** da programma tramite la funzione `open()`, emergono alcune particolarità rispetto ai file ordinari. In primo luogo, un programma **non può aprire una FIFO** con il flag `O_RDWR`, cioè per lettura e scrittura simultaneamente, poiché il comportamento risultante è **indefinito**. Tuttavia, questa limitazione è poco rilevante perché normalmente le FIFO sono utilizzate per trasferire dati in **una sola direzione**, rendendo inutile un'apertura bidirezionale.

Quando si scrive su una FIFO, i dati vengono **accodati**, mentre una lettura recupera **sempre i dati dall'inizio** della FIFO. Questo comportamento riflette la logica **First In, First Out**, tipica delle pipe.

Un'importante distinzione rispetto all'apertura di file normali riguarda l'uso dell'opzione `O_NONBLOCK` nel parametro `oflag` di `open()`. Questo flag modifica non solo il comportamento della chiamata `open()`, ma anche quello delle **successive operazioni di lettura e scrittura** sul descrittore di file ottenuto.

Le modalità valide per aprire una FIFO includono quattro combinazioni:

- `open(const char *path, O_RDONLY);`

blocca l'esecuzione fino a che un altro processo non apre la FIFO in scrittura.

- `open(const char *path, O_RDONLY | O_NONBLOCK);`

ritorna immediatamente, anche se nessun processo ha aperto la FIFO in scrittura.

- `open(const char *path, O_WRONLY);`

si blocca finché non viene aperta in lettura da un altro processo.

- `open(const char *path, O_WRONLY | O_NONBLOCK);`

ritorna subito; tuttavia, se nessun processo ha aperto la FIFO in lettura, restituisce **-1 e un errore**, e la FIFO non è disponibile. Se invece è già aperta in lettura, si ottiene un descrittore utilizzabile per scrivere.

Questi meccanismi rendono l'uso delle FIFO particolarmente adatto per la **comunicazione asincrona** e **coordinata** tra processi.

Esempio

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#define FIFO_NAME "/tmp/my_fifo"
int main(int argc, char *argv[]){
    int res;
    int open_mode = 0;
    int i;
    if (argc < 2) {
        printf("Uso: %s <combinazioni di O_RDONLY O_WRONLY O_NONBLOCK>\n", argv[0]);
        exit(1);
    }
```

Questo esempio mostra un programma in C che dimostra l'apertura di una **FIFO** utilizzando la funzione **open()** e diverse modalità di accesso passate come argomenti da linea di comando. Il nome della FIFO è definito come `"/tmp/my_fifo"`.

All'avvio, il programma verifica se l'utente ha fornito almeno un argomento, altrimenti stampa un messaggio di utilizzo e termina.

```

/* Impostiamo il valore di open_mode dagli argomenti. */
for(i = 1; i < argc; i++) {
    if (strcmp(++argv, "O_RDONLY", 8)==0) open_mode |= O_RDONLY;
    if (strcmp(*argv, "O_WRONLY", 8)==0) open_mode |= O_WRONLY;
    if (strcmp(*argv, "O_NONBLOCK", 10)==0) open_mode |= O_NONBLOCK;
}
/* Se la FIFO non esiste la creiamo. Poi viene aperta */
if (access(FIFO_NAME, F_OK) == -1) {
    res = mkfifo(FIFO_NAME, 0777);
    if (res != 0) {
        printf("Non posso creare la FIFO %s\n", FIFO_NAME);
        exit(1);
    }
}

```

```

#include <unistd.h>
int access(const char *pathname, int mode);
/* Restituisce 0 se OK, -1 in caso di errore */
Verifica l'accessibilità del real user
Mode può essere R_OK, W_OK, X_OK, F_OK.

```

Gli argomenti specificano le modalità di apertura, che possono essere **O_RDONLY**, **O_WRONLY** e **O_NONBLOCK**. Questi sono analizzati tramite `strcmp()` e combinati con l'operatore **bitwise OR** per costruire il flag `open_mode`.

Se la FIFO non esiste, viene creata con **permessi 0777** grazie alla funzione `mkfifo()`, ma soggetta alla **umask** del sistema. L'esistenza della FIFO viene controllata usando la chiamata `access()` con il flag **F_OK**, che restituisce 0 se il file esiste e -1 altrimenti.

```

printf("Processo %d apre la FIFO\n", getpid());
res = open(FIFO_NAME, open_mode);
printf("Risultato processo %d: %d\n", getpid(), res);
sleep(5);
if (res != -1) close(res);
printf("Processo %d terminato\n", getpid());
exit(0);
}

```

Una volta che la FIFO è pronta, il programma stampa il PID del processo e tenta di aprire la FIFO con `open()`, usando il `open_mode` specificato. Stampa quindi il risultato dell'operazione e attende 5 secondi (con `sleep(5)`), per simulare un'attività o per consentire l'interazione con altri processi.

Infine, se la FIFO è stata aperta correttamente (`res != -1`), viene **chiusa con `close()`**, e il programma stampa un messaggio finale prima di terminare.

Questo codice è utile per testare il comportamento delle **quattro modalità di apertura** delle FIFO, evidenziando differenze tra apertura bloccante e non bloccante e tra lettura e scrittura, in scenari reali di comunicazione tra processi.

NB:

```

/* Impostiamo il valore di open_mode dagli argomenti. */
for(i = 1; i < argc; i++) {
    if (strncmp(++argv, "O_RDONLY", 8)==0) open_mode |= O_RDONLY;
    if (strncmp(*argv, "O_WRONLY", 8)==0)    open_mode |= O_WRONLY;
    if (strncmp(*argv, "O_NONBLOCK", 10)==0) open_mode |= O_NONBLOCK;
}

```

Questa parte del codice serve a **interpretare gli argomenti passati da linea di comando** e a costruire il valore del flag `open_mode`, che verrà poi usato nella chiamata a `open()` per aprire la FIFO.

```

for(i = 1; i < argc; i++) {

```

Si inizia a leggere gli argomenti **a partire da `argv[1]`** (cioè escludendo il nome del programma `argv[0]`) e si prosegue finché ci sono argomenti.

```

if (strncmp(++argv, "O_RDONLY", 8)==0) open_mode |= O_RDONLY;

```

Qui si confronta l'argomento corrente con la stringa `"O_RDONLY"`. Se corrisponde, si **aggiunge il flag `O_RDONLY`** al valore `open_mode` usando l'operatore **bitwise OR** (`|=`). Questo serve per **combinare più flag** nel caso siano specificati più argomenti.

Stessa cosa avviene per:

```

if (strncmp(*argv, "O_WRONLY", 8)==0) open_mode |= O_WRONLY;
if (strncmp(*argv, "O_NONBLOCK", 10)==0) open_mode |= O_NONBLOCK;

```

Quindi, se esegui il programma così:

```

./programma O_WRONLY O_NONBLOCK

```

Il ciclo riconosce entrambe le stringhe e costruisce `open_mode` come:

```

open_mode = O_WRONLY | O_NONBLOCK;

```

Nota tecnica: l'uso di `++argv` nella prima riga modifica direttamente il puntatore `argv` avanzandolo, il che può essere un po' **confusionario o rischioso**. Sarebbe più chiaro e sicuro usare:

```

for (i = 1; i < argc; i++) {
    if (strncmp(argv[i], "O_RDONLY", 8) == 0) open_mode |= O_RDONLY;
    ...
}

```

Così si evitano effetti collaterali sulla variabile `argv`.

L'effetto pratico delle combinazioni dei flag **O_RDONLY**, **O_WRONLY** e **O_NONBLOCK** nell'apertura di una FIFO tramite il programma d'esempio precedentemente analizzato.

Il programma permette all'utente di **specificare da riga di comando** le modalità di apertura della FIFO tra quelle consentite. Se la FIFO non esiste, viene creata grazie a `access()` e `mkfifo()`. Non viene eliminata alla fine dell'esecuzione perché non si ha modo di sapere se un altro programma la stia utilizzando.

Esecuzione senza O_NONBLOCK

```
$ ./a.out O_RDONLY &
[1] 152
Processo 152 apre la FIFO
$ ./a.out O_WRONLY
Processo 153 apre la FIFO
Risultato processo 152: 3
Risultato processo 153: 3
Processo 152 terminato
Processo 153 terminato
```

Quando il lettore apre la FIFO con **O_RDONLY** e lo scrittore con **O_WRONLY**, entrambi **si bloccano** finché l'altro processo non effettua la chiamata corrispondente a `open()`. In questo modo, i due processi sono **sincronizzati**: il lettore può partire per primo e attendere lo scrittore, e solo quando entrambi sono pronti, la FIFO viene aperta con successo da entrambi. È un comportamento **bloccante**, utile per la sincronizzazione implicita tra processi.

Esecuzione con O_NONBLOCK

```
$ ./a.out O_RDONLY O_NONBLOCK &
[1] 160
Processo 160 apre la FIFO
Risultato processo 160: 3
$ ./a.out O_WRONLY
Processo 161 apre la FIFO
Risultato processo 161: 3
Processo 160 terminato
Processo 161 terminato
```

Nel caso in cui il lettore apra la FIFO con **O_RDONLY | O_NONBLOCK**, la chiamata a `open()` **ritorna immediatamente**, anche se nessuno ha ancora aperto la FIFO in scrittura. Lo stesso avviene se lo scrittore usa **O_WRONLY**, purché la FIFO sia già aperta da un lettore.

Questa modalità **non bloccante** è utile per situazioni in cui non si vuole che un processo attenda indefinitamente un altro, ma comporta la necessità di **gestire a livello applicativo** l'eventuale assenza del partner di comunicazione.

In sintesi:

- Senza **O_NONBLOCK**, i processi si **sincronizzano automaticamente** tramite la FIFO.
- Con **O_NONBLOCK**, i processi proseguono indipendentemente, ma richiedono **logica aggiuntiva** per gestire correttamente la comunicazione.

Di seguito è spiegato il comportamento delle chiamate di sistema **read()** e **write()** sulle FIFO, distinguendo tra modalità **bloccante** e **non bloccante**.

FIFO in modalità bloccante

Quando la FIFO è aperta in modalità predefinita (senza **O_NONBLOCK**):

- Una chiamata a **read()** su una FIFO **vuota** si blocca finché un processo non scrive dei dati. Se però nessun processo ha aperto la FIFO in scrittura, `read()` restituisce **0**, segnalando che non c'è nulla da leggere.
- Una chiamata a **write()** si blocca finché non ci sarà spazio disponibile nel buffer per i dati da scrivere, ma solo se la FIFO è stata aperta in lettura. Se non c'è alcun lettore, `write()` genera un **segnale SIGPIPE**, che può terminare il processo se non gestito.

FIFO in modalità non bloccante (**O_NONBLOCK**)

L'aggiunta del flag **O_NONBLOCK** cambia radicalmente il comportamento:

Lettura (`read()`)

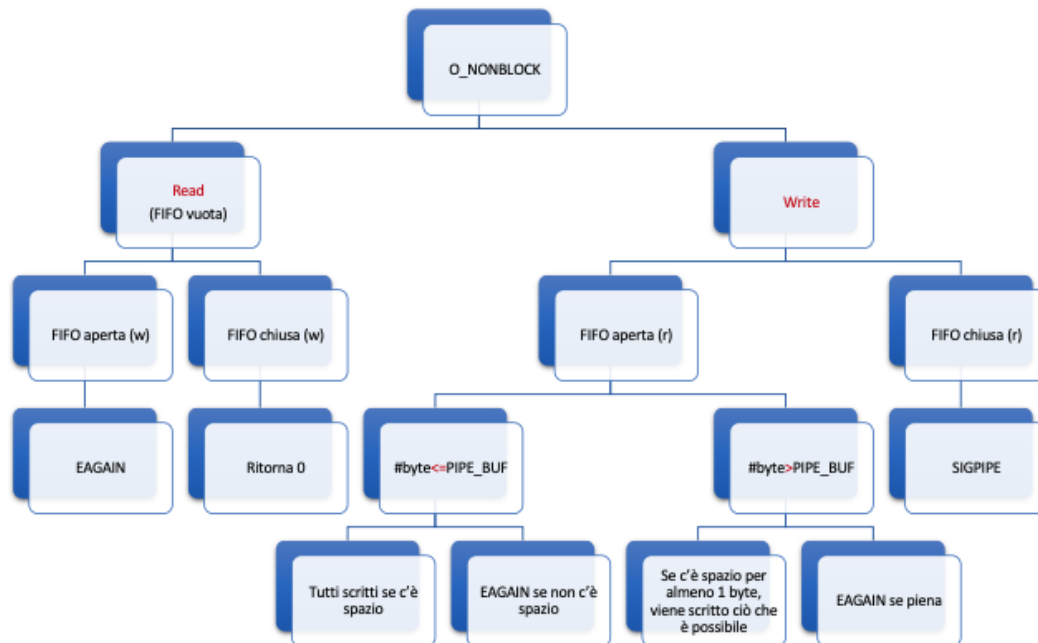
- Se la FIFO è vuota ma **aperta in scrittura**, `read()` **non si blocca**, ma restituisce l'errore **EAGAIN**.
- Se la FIFO non è aperta in scrittura, `read()` restituisce **0**, come in modalità bloccante.

Scrittura (`write()`)

- Se **nessun processo ha aperto la FIFO in lettura**, viene generato **SIGPIPE**.
- Se la FIFO è **aperta in lettura**:
 - Se si tenta di scrivere un numero di byte $\leq$ **PIPE_BUF**, e c'è spazio per tutti, i dati vengono scritti interamente.
 - Se non c'è spazio sufficiente, `write()` **non si blocca**, ma restituisce **EAGAIN**.
 - Se si vogliono scrivere più byte di **PIPE_BUF**, e c'è **almeno 1 byte di spazio**, verrà scritto quanto possibile, e `write()` ritorna il numero di byte scritti.
 - Se non c'è spazio nemmeno per un byte, `write()` fallisce con **EAGAIN**.

La modalità **bloccante** è più semplice da gestire e favorisce la **sincronizzazione automatica** tra i processi. La modalità **non bloccante** offre invece maggiore controllo e flessibilità, ma richiede una **gestione esplicita** degli errori e dei flussi di comunicazione.

Lettura e scrittura con O_NONBLOCK impostato



Questa mappa concettuale riassume in modo visivo il comportamento delle chiamate **read()** e **write()** su una **FIFO aperta con O_NONBLOCK**, distinguendo i vari casi a seconda dello stato della FIFO e delle condizioni operative.

Nodo centrale: O_NONBLOCK

L'intero schema si basa sull'ipotesi che il file FIFO sia aperto in modalità **non bloccante**, cioè con il flag **O_NONBLOCK**.

Sezione Read (lettura da FIFO vuota)

- **FIFO aperta in scrittura (w)**
Se un processo scrive sulla FIFO, ma al momento della `read()` non ci sono dati disponibili, la chiamata **non si blocca**, ma restituisce **EAGAIN**, segnalando che i dati non sono pronti.
- **FIFO chiusa in scrittura (w)**
Se nessun processo ha aperto la FIFO in scrittura, `read()` restituisce **0**, indicando che non c'è più nessuno che scriverà dati.

Sezione Write (scrittura su FIFO)

- **FIFO chiusa in lettura (r)**
Se nessun processo ha aperto la FIFO in lettura, una chiamata a `write()` genera **SIGPIPE**, che solitamente termina il processo a meno che il segnale non venga gestito.
- **FIFO aperta in lettura (r)**
 - **#byte ≤ PIPE_BUF**
Se i byte da scrivere sono pochi (fino alla dimensione di `PIPE_BUF`), e c'è abbastanza spazio nel buffer, **tutti i byte vengono scritti**.
Se invece **non c'è spazio**, `write()` restituisce ****EAGAIN**.
 - **#byte > PIPE_BUF**
In questo caso, anche se c'è spazio solo per un byte, il kernel ne scrive quanti può e ritorna il numero effettivamente scritto.
Se la FIFO è **completamente piena**, `write()` restituisce **EAGAIN**.

Questo schema riassume con chiarezza le conseguenze dell'uso di `O_NONBLOCK`: le chiamate **non attendono**, ma reagiscono immediatamente, restituendo un errore o un valore specifico a seconda delle condizioni della FIFO. È quindi essenziale per i programmi che lo usano **gestire attivamente questi casi**, evitando di assumere che l'operazione vada sempre a buon fine.

Esempio: comunicazione tra due processi con FIFO (produttore-consumatore)

fifo1.c

```
#include <unistd.h>

...

#define FIFO_NAME "/tmp/my_fifo"
#define BUFFER_SIZE PIPE_BUF
#define TEN_MEG (1024 * 1000 * 10)

int main(){
    int pipe_fd;
    int res;
    int open_mode = O_WRONLY;
    int bytes_sent = 0;
    char buffer[BUFFER_SIZE];
    if (access(FIFO_NAME, F_OK) == -1) {
        res = mkfifo(FIFO_NAME, 0777);
        if (res != 0) {
            printf("Could not create fifo %s\n", FIFO_NAME);
            exit(-1);
        }
    }

    printf("Process %d opening FIFO O_WRONLY\n", getpid());
    pipe_fd = open(FIFO_NAME, open_mode);
    printf("Process %d result %d\n", getpid(), pipe_fd);

    if (pipe_fd != -1) {
        while(bytes_sent < TEN_MEG) {
            // supponiamo che buffer sia riempito altrove
            res = write(pipe_fd, buffer, BUFFER_SIZE);
            if (res == -1) {
                printf("Write error on pipe\n");
                exit(1);
            }
            bytes_sent += res;
        }
        close(pipe_fd);
    }
    else
    {
        exit(1);
    }
    printf("Process %d finished\n", getpid());
    exit(1);
}
```

Produttore (fifo1.c): crea una FIFO se richiesto, poi vi scrive i dati quanto prima.

fifo2.c

```
#include <string.h>
#include <fcntl.h>
#include <limits.h>
#include <sys/types.h>
#include <sys/stat.h>
#define FIFO_NAME "/tmp/my_fifo"
#define BUFFER_SIZE PIPE_BUF

int main(){
    int pipe_fd;
    int res;
    int open_mode = O_RDONLY;
    char buffer[BUFFER_SIZE];
    int bytes_read = 0;
    printf("Process %d opening FIFO O_RDONLY\n", getpid());
    pipe_fd = open(FIFO_NAME, open_mode);
    printf("Process %d result %d\n", getpid(), pipe_fd);

    if (pipe_fd != -1) {
        do {
            res = read(pipe_fd, buffer, BUFFER_SIZE);
            bytes_read += res;
        } while (res > 0);
        close(pipe_fd);
    }
    else {
        exit(-1);
    }
    printf("Process %d finished,%d bytes read\n",getpid(),bytes_read);
    exit(0);
}
```

Questo esempio illustra una tipica **comunicazione tra processi** utilizzando una **FIFO in modalità bloccante**, secondo il modello **produttore-consumatore**. Il processo produttore (`fifo1.c`) ha il compito di creare la FIFO, se non esiste già, e di scrivere al suo interno un ampio volume di dati. Il processo consumatore (`fifo2.c`) si occupa invece di leggere questi dati e smaltirli.

Entrambi i programmi aprono la FIFO rispettivamente in modalità **O_WRONLY** e **O_RDONLY**, quindi in modalità **bloccante**. Questo significa che il produttore, una volta eseguito, **rimane in attesa** fino a quando il consumatore non apre la FIFO per la lettura. Solo a quel punto la chiamata `open()` del produttore ha successo e il flusso di scrittura può iniziare. In parallelo, il consumatore riceve i dati via `read()` e li elimina dal buffer FIFO.

Il produttore scrive ripetutamente porzioni di dati, utilizzando un buffer della dimensione di **PIPE_BUF**, fino a raggiungere il volume di **10 megabyte**. Il consumatore legge i dati in un ciclo, accumulando il numero di byte letti, finché il produttore non chiude la FIFO.

```
$ ./fifo1 &
[1] 375
Process 375 opening FIFO O_WRONLY
$ ./fifo2
Process 377 opening FIFO O_RDONLY
Process 375 result 3
Process 377 result 3
Process 375 finished
Process 377 finished, 10485760 byte read
```

L'esecuzione dei due programmi dimostra un comportamento perfettamente **sincronizzato**, reso possibile proprio dal fatto che la FIFO opera in **modalità bloccante**, favorendo così la **coordinazione implicita** tra i processi. L'output conferma che entrambi i processi hanno terminato correttamente il trasferimento dati, con lettura e scrittura perfettamente bilanciate.

Comunicazione Client-Server con FIFO

Nel modello descritto, il server crea una **FIFO condivisa** attraverso cui riceve le richieste dei client. Ogni client può scrivere nella FIFO, purché i messaggi siano **inferiori o uguali a PIPE_BUF** in termini di dimensione. Questo garantisce che ciascuna scrittura sia **atomica**, evitando che i dati di più client vengano mescolati all'interno della FIFO.

Il server, eseguendo una lettura bloccante su questa FIFO, riceve le richieste una alla volta. L'uso della modalità **bloccante** consente una **sincronizzazione implicita** tra il server e ogni client: un client si blocca sulla scrittura finché il server non legge, e viceversa.

La fase più complessa riguarda la **restituzione della risposta** ai client. Poiché più client condividono la stessa FIFO per inviare le richieste, non possono utilizzare quella stessa pipe per ricevere risposte personalizzate. La soluzione proposta consiste nell'associare a ciascun client una **seconda FIFO dedicata alla risposta**. Il nome di questa FIFO può essere generato dinamicamente usando, ad esempio, il **PID del client**, incluso nel messaggio inviato al server. In questo modo, il server può creare o aprire una FIFO specifica e inviare la risposta al client che l'ha richiesta, garantendo una comunicazione **bidirezionale e privata**.

Codice client/server con Pipe FIFO lez. 13 slide 30-37

Gestione del Ciclo di Vita e Persistenza del Server nella Comunicazione FIFO

*Funzionamento **client-server con FIFO** e comportamento del sistema in modalità **bloccante** e i meccanismi di sincronizzazione impliciti tra i processi.*

Il **server** crea la propria FIFO in **sola lettura** e rimane **bloccato** finché un **client** non la apre in scrittura. Solo allora il server si sblocca e può iniziare a ricevere dati. Una **sleep temporanea** viene inserita a fini dimostrativi, per far sì che più client possano avviarsi e accodarsi correttamente (questa tecnica non è raccomandata nelle applicazioni reali).

Nel frattempo, il **client**, una volta aperta la FIFO del server, crea una **FIFO personale**, identificata tramite il proprio **PID**, che sarà usata per ricevere la risposta. Dopo averla creata, il client invia i dati al server e si **blocca** in lettura sulla propria FIFO in attesa della risposta.

Il **server**, ricevuti i dati, li elabora (nell'esempio li converte in maiuscolo) e apre la FIFO del client in **sola scrittura**. Scrivendo la risposta, il server **sblocca il client**, che può così leggerla e completare il proprio lavoro.

Questo ciclo si ripete per ogni client finché **l'ultimo client chiude la FIFO del server**. In quel momento, la successiva chiamata `read()` del server su quella FIFO restituisce **0**, indicando che nessun processo ha più la FIFO aperta in scrittura.

In un'applicazione **reale**, dove il server deve rimanere attivo per accettare ulteriori client anche dopo che uno ha terminato, è necessario **modificare il comportamento**. Due soluzioni comuni sono: aprire la FIFO anche in **scrittura**, per evitare che `read()` restituisca 0, oppure **riaprire la FIFO** ogni volta che viene chiusa da tutti i client, per continuare ad accettare nuove connessioni. In entrambi i casi, si garantisce che il server resti disponibile e pronto a gestire nuove richieste.

Riassunto in poche parole

Il client apre la FIFO del server per inviare un messaggio, e crea la sua FIFO per ricevere la risposta; il server legge dalla sua FIFO e poi scrive la risposta nella FIFO personale del client — tutto questo avviene con aperture e chiusure che **sincronizzano automaticamente** i due.

